

29. Datové a řídicí struktury imperativních programovacích jazyků

29. Datové a řídicí struktury imperativních jazyků

SZZ okruh 29 (FIT BUT). Stavební kameny imperativního programování.

Shrnutí

Řídicí struktury

- **Sekvence** (příkazy po sobě), **selekce** (if/switch), **iterace** (for/while/do-while) — trojice, která činí jazyk **Turing-úplným**.
- **Funkce** (rozklad na podprogramy), **rekurze** (definice pomocí sebe sama; každou lze přepsat na iteraci + zásobník/frontu).
- Cykly: while (podmínka napřed), do-while (tělo aspoň jednou), for (n-krát).

Datové typy a struktury

- **Datový typ** = množina hodnot + operace. Dělení: standardní/uživatelské, ordinální/neordinální, jednoduché/strukturované.
- **Strukturované**: homogenní (pole) × heterogenní (struktura/record); statické × dynamické.
- Statická vs. **dynamická alokace** paměti.

Abstraktní datové typy (ADT)

- **ADT** = množina hodnot + operace, **abstrahuje od implementace** (co, ne jak).
- **Spojový seznam** (jedno-/obousměrný, kruhový), **zásobník** (LIFO — DFS, vyhodnocení výrazů), **fronta** (FIFO — BFS, hromadná obsluha), **binární strom** (průchody pre/in/post-order; vyvážené AVL, red-black).
- **Vyhledávací tabulka / hash tabulka** — klíč → index mapovací funkcí; **kolize** řešeny **implicitním** (otevřená adresace) nebo **explicitním zřetězením** (seznam/strom); load factor ~70 %.

Algoritmy nad těmito strukturami viz [[okruhy/30-vyhledavani-razeni]]; složitost viz [[okruhy/32-slozitost-algoritmu]]; ADT a zapouzdření viz [[okruhy/40-objektova-orientace]].

Související syntéza

- [[synthesis/zasobnik-napric-obory | Zásobník napříč obory]] — syntéza
- [[synthesis/vyhledavaci-b-stromy | Vyhledávací stromy × paměťová hierarchie]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Datové struktury □ [[#Datové typy a struktury]] - *Statická vs. dynamická alokace?* □ Statická = pevná velikost při překladu; dynamická = za běhu (halda, ukazatele). - *Pole vs. spojový seznam v paměti?* □ Pole souvislý blok (přímý přístup $O(1)$); seznam rozptýlené uzly s ukazateli (vkládání $O(1)$, přístup $O(n)$).

ADT □ [[#Abstraktní datové typy (ADT)]] - *Co je ADT a proč?* □ Množina hodnot + operace bez detailů implementace; zvyšuje abstrakci, snižuje provázanost. - *Zásobník vs. fronta?* □ LIFO (push/pop/top) × FIFO (enqueue/dequeue); použití DFS × BFS. - *Binární strom — průchody?* □ pre-order (uzel,L,P), in-order (L,uzel,P), post-order (L,P,uzel); vyvážení (AVL, red-black) brání degradaci na seznam. - *Hash tabulka, kolize?* □ Klíč □ index; kolize řeší implicitní (otevřená adresace) nebo explicitní zřetězení (seznam/strom).

Řídící struktury a rekurze □ [[#Řídící struktury]] - *Sekvence/selekce/iterace?* □ Tři základní konstrukce strukturovaného programování (□ Turing-úplnost). - *Cyklus vs. rekurze?* □ Každou rekurzi lze přepsat na iteraci (s explicitním zásobníkem); rekurze přirozená u stromů.

Plné znění (ke studiu)

Řídící struktury

- **Sekvence** - umožňuje provádět **příkazy (přiřazení, volání funkce, aritmetické operace, ...)** **jeden po druhém** (řetězení příkazů). Obecněji **umožňuje provádění podprogramů jeden po druhém**.
- **Selekce** - Umožňuje **volbu mezi bloky kódu**, které budou provedeny na základě hodnoty booleovského výrazu (**if - else if - else, switch, ...**). Obecněji **umožňuje vybrat jeden ze dvou podprogramů**, který bude prováděn, na základě hodnoty booleovského výrazu.
- **Iterace** - **Opakování bloku kódu** na základě hodnoty booleovského výrazu (**for, while, repeat, do while, goto, ...**). Obecněji umožňuje **opakování vykonávání podprogramu**.

Všechny programovací jazyky, které podporují tyto řídicí struktury (a jsou vybaveny **nekonečným** sekundárním úložištěm - tento fakt je většinou ignorován) jsou **Turing Complete**. Existují i jazyky, které mají pouze jedinou instrukci a jsou Turing Complete (pomocí této instrukce je však simulována manipulace s pamětí, větvení a iterace). Tato trojice je také **základem strukturovaného programování**.

[[media/szz-29/media/image4.png]]

Výraz/příkaz

Například **aritmetický výraz**, prostý **výskyt konstanty či proměnné, funkční volání, přiřazení, ...** Výrazový příkaz se získá **ukončením výrazu patřičným symbolem** (v C, C++, C# aj. je to středník, v Pythonu je to konec řádku). Zvláštním případem je prázdný příkaz (samostatný středník).

Podmíněný příkaz if-else

Umožňuje rozhodnutí na základě hodnoty proměnné (**true/false**), jestli **bude** kód vykonán (true), nebo **nebude** (false). Respektive v případě nepravdy (false) se vykoná **else** větev, pokud existuje.

Podmíněný příkaz switch

Slouží k **větvení** podle hodnoty **celočíslného výrazu, řetězce, hodnoty výčtu** atd. Řízení programu přechází do úseku kódu jehož konstantní hodnota (která jej uvádí - hodnota **case**) odpovídá hodnotě proměnné, podle které se přepíná ("switchuje" v příkazu **switch**). Switch také umožňuje provést výchozí chování (větev **default**), pokud žádný z **case** nevyhovuje.

Cykly

Umožňují několikanásobné (**for** cyklus) opakování kódu. Nekonečné cykly (**while(true)**) je možné obvykle ukončit příkazem **break**. Přeskočení zbytku nynější iterace je možné obvykle příkazem **continue**.

- **while** - Vhodné například pro předem neznámý počet provádění. Vyhodnocuje se poprvé podmínka, tedy **tělo nemusí být vykonáno**.
- **do while (repeat until)** - Tělo je **provedeno alespoň jednou**.
- **for** - Tělo se provede **přesně n-krát**, kde **n** je přesně stanovený počet iterací, pokud neuvažujeme modifikaci ve vyšších programovacích jazycích na podmínku.
- **goto** - v případě skoku na **návěští**, které **goto předchází**. Skákat lze pouze lokálně (tedy nelze skočit pryč z funkce - na to je např. longjump v C - to je nebezpečné narozdíl od goto).

Složený příkaz (blok)

Posloupnost libovolných příkazů (**včetně dalších bloků**) s vlastním kontextem. Často se **syntakticky odlišuje** (**{}** v C, **begin end.** v Pascalu, **odsazení** v Python). **Proměnné** zde definované mají často **platnost pouze v tomto bloku - lokální proměnné**.

Funkce

Umožňují rozložení složitého problému na jednodušší **podprogramy**, které mají jedno vstupní místo (obvykle standardizovaným způsobem). **Deklarace** funkce se skládá z **návratového typu, názvu funkce a seznamu parametrů**. Deklaraci je často možné vynechat a funkci přímo definovat. Funkce jsou jedním z hlavních předpokladů pro **strukturované jazyky**.

Rekurze

Metoda definování určitého objektu pomocí sebe sama.

- Umožňuje definovat **nekonečnou množinu** objektů **konečným popisem - rekurentní definice**.
- Rekurzivní struktura dat (lineární seznam).
- Rekurzivní struktura algoritmu (DFS, InOrderTraversal).
- Rekurzivní volání funkce – **funkce je volána v těle sebe samé** (obecně se tomuto případu snažíme vyhnout, **jednoduché** na implementaci, ale **nebezpečné**).
 - Každou rekurzi lze **vždy přepsat na iterativní vykonávání**, případně s využitím vhodné **datové struktury** (fronta, zásobník atd.).

Datové struktury

Datová struktura určuje konkrétní způsob **organizace dat v paměti počítače**.

Datový typ

Určuje **množinu hodnot** a **množinu operací nad těmito hodnotami**. Typ proměnné se zavádí v **její deklaraci**. Je určen:

- názvem,
- množinou hodnot, kterých může nabývat a

- množinou operací, které mohou být prováděny nad hodnotami.

Dělení datových typů (dle několika způsobů):

- **standardní** (int, double, char, string, boolean),
- **definované uživatelem** (enum, pole, záznam, množina, soubor, ukazatel),
- **ordinální** - prvky daného typu mají jasně stanovené pořadí. Každý prvek má předchůdce a následovníka (integer, char, ...),
- **neordinální** (string, real, pointer, ...),
- **jednoduché/základní** - nemají vnitřní strukturu, s proměnnou lze pracovat jako s celkem (integer, char, real, ...),
- **strukturované** - hodnoty mají vnitřní strukturu (členění na komponenty), s proměnnou strukturovaného typu lze pracovat buď jako s celkem, nebo s jeho jednotlivými komponenty (array, struct, record, ...).

Strukturovaný datový typ

Sestává z komponent jiného (dříve definovaného) typu = kompoziční typ. Má strukturovanou hodnotu a musí mít definované hodnoty všech komponent.

- **Homogenní** - Všechny komponenty (položky) jsou **stejného typu - pole**.
- **Heterogenní** - Komponenty (složky) jsou **rozdílného typu - struktura**.
- **Statický** - **Nemůže měnit** v průběhu výpočtu **počet komponent ani způsob uspořádání**.
- **Dynamický** - **Může měnit** v průběhu **počet komponent i způsob uspořádání**.

Pole Homogenní ortogonální (pravoúhlý) datový typ. Typ je **specifikovaný velikostí svých dimenzí a komponentním typem**. K prvkům se přistupuje pomocí **identifikátoru pole** a **indexu prvku**.

- **Vektor** - jednorozměrné pole.
- **Matice** - dvourozměrné pole.

Řetězec Strukturovaný homogenní datový typ. Položky mají **typ znak** (char). Má vlastnosti (a často je tak i implementován) jako **jednorozměrné pole znaků**, ale ve většině vyšších programovacích jazyků umožňuje více operací (např. porovnání, konkatenace, ...).

Záznam (record/struct) Strukturovaný statický heterogenní datový typ. Komponenty mohou být libovolného datového typu. Jména a počet komponent jsou dány při definici typu a nemohou se měnit za běhu programu.

Abstraktní datové typy

Při tvorbě reálných aplikací lze využít **obecného modelu datové struktury** vyjádřeného pomocí abstraktního datového typu:

- určíme použité **datové komponenty**,
- určíme **operace** a jejich **vlastnosti**,
- **abstrahujeme** od způsobu implementace.

ADT zdůrazňuje, co dělá, ale potlačuje, jak to dělá. Smyslem je **zvýšit** datovou **abstrakci** a **snížit** algoritmickou **složitost** programu. ADT je definován **množinou hodnot**, kterých smí nabýt každý prvek tohoto typu, a **množinou operací** nad tímto datovým typem.

Spojový seznam (linked list) Homogenní, lineární, dynamická struktura. Prvkem seznamu může být **jakýkoliv typ** (také strukturovaný). Seznam může být prázdný.

- **Ekvivalence** - Když jsou oba seznamy prázdné a nebo když se rovnají jejich první prvky a také jejich zbytky (**rekurentní definice pomocí rekurze**).
- **Jednosměrný** - Prvek ukazuje pouze na svého **následovníka**.

[[media/szz-29/media/image5.png]]

- **Dvousměrný** - Prvek ukazuje na svého **následovníka** i **předchůdce**.

[[media/szz-29/media/image2.png]]

- **Kruhový jednosměrný** - Jednosměrný, kde **poslední** prvek má za **následovníka** první prvek.

[[media/szz-29/media/image6.png]]

- **Kruhový dvousměrný** - Dvousměrný, kde **poslední prvek** má za **následovníka** první prvek a **první prvek** má za **předchůdce** poslední prvek.

Operace nad seznamem:

- **inicializace**,
- **vložení** prvku na **začátek seznamu** (lze vložit do prázdného seznamu),
- **získání** hodnoty **prvního** prvku seznamu,
- **odstranění** prvního prvku seznamu,
- **aktivace prvního** prvku,
- **aktivace následujícího** nebo předcházejícího (u dvousměrného) prvku,
- **získání** hodnoty **aktivního** prvku,
- **změna** hodnoty **aktivního** prvku,
- **vložení** prvku **za/před aktivní** (před pouze u dvousměrného) prvek
- **test na prázdnost**,
- **test na aktivní prvek**.

Zásobník

Homogenní, lineární, dynamická struktura typu **LIFO (Last in - First out)**. Využívá se pro reverzaci pořadí, konstrukci rekurzivních programů bez rekurze - **DFS**. Implicitně používá zásobník operačního systému každý program při volání funkcí. Lze implementovat pomocí **pole** nebo **spojuvého seznamu**.

Použití:

- Vyčíslování aritmetických výrazů v postfixové notaci
- Převod z infixové do postfixové notace

Operace se zásobníkem:

- **Init** - Vytvoří prázdný zásobník.
- **Push** - Vloží prvek na vrchol zásobníku.
- **Pop** - Vyjme prvek z vrcholu zásobníku.
- **Top** - Přečte hodnotu z vrcholu zásobníku.
- **Empty** - True pokud je prázdný.

Fronta

Dynamická, homogenní a lineární struktura typu **FIFO (First in - First out)**. Na jedné straně se přidává (konec fronty) a na druhé se čte a odebírá – obsluhuje (začátek fronty). Využívá se např. v **systémech hromadné obsluhy**, implementaci **BFS**, návrhový vzor **producent konzument**. Lze implementovat pomocí **pole** nebo **spojového seznamu**. Existují prioritní fronty, kde mohou prvky s vyšší prioritou předbíhat.

- **Init** - Vytvoří prázdnou frontu.
- **Enqueue** - Vloží prvek na konec fronty.
- **Dequeue** - Výjme prvek ze začátku.
- **Front** - Přečte hodnotu ze začátku fronty.
- **Empty** - True pokud je prázdná.

Binární strom

Homogenní dynamická struktura. Rekurentní definice:

Binární strom je **buď prázdný** nebo se sestává z **jednoho uzlu** zvaného **kořen** a **dvou podstromů** (levého a pravého) a oba podstromy mají vlastnost stromu.

[[media/szz-29/media/image7.png]]

- **Váhově vyvážený** - Pokud pro **všechny jeho uzly** platí, že **počty uzlů** jejich **levého** podstromu a **pravého** podstromu **se rovnají nebo se liší právě o 1**.
- **Výškově vyvážený** - Když pro **všechny jeho uzly** platí, že **výška levého** podstromu **se rovná výšce pravého** podstromu nebo se **liší právě o 1**. [[media/szz-29/media/image1.png]]

Při zajištění vyváženosti **nemůže dojít k degradaci** stromu na seznam. Samovyvažující se stromy: **red-black tree** (červený uzel nemá červeného následníka a na každé cestě z libovolného uzlu k listu je stejný počet černých uzlů), **AVL tree** (uzly mají váhu - **0: zcela vyvážený** uzel, **-1: výška levého** podstromu je o jedna **větší**, **1: výška pravého** podstromu je o jedna **větší**, pokud dojde ke změně váhy na **-2/2**, je nutné situaci napravit - **rotace**).

Průchody stromem

- **Preorder** - aktuální uzel, levý podstrom, pravý podstrom.
- **Inorder** - levý podstrom, aktuální uzel, pravý podstrom.
- **Postorder** - levý podstrom, pravý podstrom, aktuální uzel.
- **InvPreOrder** - aktuální uzel, pravý podstrom, levý podstrom.
- **InvInOrder** - pravý podstrom, aktuální uzel, levý podstrom.
- **InvPostOrder** - pravý podstrom, levý podstrom, aktuální uzel.

[[media/szz-29/media/image3.png]]

Vyhledávací tabulka (Look-up table)

Homogenní, obecně **dynamická struktura**, ve které má **každá položka** zvláštní složku - **klíč**. Klíč by měl být v tabulce **unikátní** (s jedinečnou hodnotou), aby bylo podle něj možné provádět (ostré) vyhledávání. V ideálním případě je vyhledávání s konstantní časovou složitostí. **NEurčuje**, jak se vyhledávání provádí: může to být seznam (sekvenční vyhledávání), pole, strom nebo hash tabulka. Cílem je vkládat, mazat a hledat podle klíče.

Tabulka s přímým přístupem Implementuje se pomocí pole, ve kterém jsou klíče mapovány na indexy pole. To vyžaduje vzájemně jednoznačné zobrazení (**bijekce**) mapující každý prvek množiny klíčů **K** do množiny indexů pole **H**. Jedná se o ideální strukturu pro vyhledávání, která je ale v praxi prakticky nepoužitelná (stejně velké pole jako počet klíčů je nereálné). Proto se používá mapovací (hashovací) funkce, která obvykle mapuje větší počet klíčů na menší počet hodnot. Vznikají tak **kolize** - dva různé klíče jsou namapovány do stejného místa (na stejný index). **Synonyma** jsou poté - dva nebo více klíčů, které jsou namapovány do téhož místa. Tabulky je vhodné navrhovat tak, aby bylo jejich očekávané naplnění **70 %-75 % (load factor)**. Problém kolizí a synonym lze řešit:

- **Implicitní zřetězení**: adresa následníka se získá pomocí funkce z adresy předchůdce (otevřená adresace). V praxi se **umístí prvek** s klíčem mapovaným na již obsazenou pozici na **první volnou pozici**, která následuje za získaným indexem. Vyhledávání se poté provádí **sekvenčně** (případně s nějakým **krokem** - jednotkový má tendenci tvořit shluky, ten se může zvětšovat, velikost tabulky by poté měla odpovídat **prvočíslu**) od této pozice, dokud se nenarazí na požadovaný prvek nebo na prázdné místo (musí se uvažovat přechod z konce na začátek, realizováno pomocí modulu).
- **Explicitní zřetězení**: adresa následníka je obsažena v jeho předchůdci (zřetězení záznamů). Lze řešit **spojovým seznamem** na každém indexu pole, položky s klíči, které se mapují na stejný index, jsou poté ukládány do těchto seznamů. Při použití nekvalitní mapovací funkce může degradovat na seznam s lineární složitostí. To lze řešit použitím **stromu místo seznamu** na každém indexu. Pak bude v nejhorším případě složitost vyhledávání logaritmická (při vyváženém stromu).

V obou případech lze tabulku v případě nutnosti zvětšit a záznamy přemapovat. Jedná se ale o složitou operaci.

Mapovací (hashovací) funkce Kvalitní mapovací funkce by měla splňovat tyto požadavky:

- Determinismus: Pro daný klíč vrátí vždy stejnou hodnotu.
- Rovnoměrné (uniformní) rozložení: Na každé místo se mapuje přibližně stejně velké množství klíčů.
- Využití celých vstupních dat: Zohlednění každého bitu, viz následující bod.
- Vyhnutí se kolizím podobných klíčů: V praxi bývá řada klíčů velice podobných. Rychlý výpočet.

Výsledek mapovací funkce musí být v rozsahu pole, do kterého jsou prvky ukládány (lze zajistit operací modulo délkou pole).

Odkazy:

- <http://dudka.cz/studyIAL>

Zdroje

- SZZ okruh 29 — studijní materiály FIT BUT (szz-29.docx). Obrázky: media/szz-29/.

Navigace: [\[\[index | • Přehled okruhů\]\]](#) · [\[\[okruhy/28-modelovani-simulace | 28. Modelování a simulace\]\]](#) · další: [\[\[okruhy/30-vyhledavani-razeni | 30. Vyhledávání a řazení\]\]](#) [□](#)